

# 整合練習 6：RFC 基礎與 Native SQL

## Lecture

**RFC ( Remote Function Call )** 機制：if01 已經教過「Remote-Enabled Module」這個 FM 屬性 ( `processingType="rfc"` ) ——這就是 RFC 機制的基礎：一支 FM 只要勾選 Remote-Enabled，就能被外部系統 ( 另一套 SAP 系統、或透過 SAP 提供的連線函式庫如 JCo/NCx 的非 SAP 系統 ) 用網路呼叫執行，語法上呼叫端完全感覺不出差異 ( `CALL FUNCTION 'xxx' DESTINATION 'yyy' ...` 才需要指定目的地，本機呼叫不需要 )。if07 會教「反過來，我方系統要主動呼叫外部系統」時要設定的 RFC Destination ( SM59 )；這題先把「RFC 到底是什麼」跟「Native SQL」這兩個各自獨立但常被放在一起講的主題釐清。

**Native SQL——繞過 Open SQL 那層資料庫無關抽象**：整個課程到目前為止，寫 SQL 都是用 Open SQL ( `SELECT ... FROM ... INTO TABLE` )，這是 ABAP 的資料庫無關層——同一段 Open SQL 語法在 Oracle / HANA / SQL Server 底下都能執行，因為 ABAP Runtime 會把它轉譯成對應資料庫的原生語法。**Native SQL** ( `EXEC SQL. ... ENDEXEC.` ) 則是直接把資料庫原生的 SQL 語法嵌進 ABAP 程式，繞過這層轉譯——官方文件明確列出兩個直接後果 ( `ABAPEXEC` 文件原文 )：

All Native SQL statements bypass table buffering, and no implicit client handling is performed.

這句話有兩層意義，第二層正是這個課程反覆出現的主題：

1. **繞過 Table Buffering**：Open SQL 對有設定緩衝 ( Buffering ) 的表，可能直接讀應用伺服器的記憶體快取而不真的問資料庫；Native SQL 一律直接問資料庫，不經過這層緩衝。
2. **沒有隱含 Client 處理**：這正是 AMDP 課程 am01 教過的同一個坑，這次換了個技術包裝再出現一次——Open SQL 的 `SELECT` 語句，ABAP 框架自動幫你加上 `WHERE mandt = sy-mandt`；Native SQL 完全沒有這層自動處理，要自己手動寫 `WHERE mandt = :sy-mandt`，不寫就會撈出所有 Client 的資料。到目前為止這已經是第三次遇到同一個模式 ( Open SQL 自動 / AMDP SQLScript 不自動 / Native SQL 不自動 ) ——規律很清楚：只要繞過 Open SQL 這層框架，Client 過濾就要自己負責。

**語法骨架** ( `EXEC SQL...ENDEXEC` 官方文件確認的語法，`:變數` 是 Host Variable 語法，把 ABAP 變數的值傳給 SQL 或接收 SQL 的結果 )：

```
EXEC SQL.  
  SELECT val1, val2  
  INTO :wa1, :wa2  
  FROM abap_docu_demo_mytab  
  WHERE val1 = :key  
ENDEXEC.  
  
IF sy-subrc = 0.  
  " 找到一筆  
ENDIF.
```

單筆查詢可以直接 `INTO :變數`，但如果預期會撈出多筆，要用游標 ( **Cursor** ) 手動 `OPEN / FETCH / CLOSE` ( 跟 AMDP 課程 am03 學過的 SQLScript 手動游標概念類似，但這裡是 ABAP 層級的游標，不是資料庫程序裡的 )：

```

EXEC SQL.
  OPEN dbcur FOR
    SELECT connid, cityfrom, cityto
    FROM spfli
    WHERE mandt = :sy-mandt AND
      carrid = :carrid
ENDEXEC.

DO.
  EXEC SQL.
    FETCH NEXT dbcur INTO :connid, :cityfrom, :cityto
  ENDEXEC.
  IF sy-subrc <> 0.
    EXIT.
  ELSE.
    " 處理這一筆
  ENDIF.
ENDDO.

EXEC SQL.
  CLOSE dbcur
ENDEXEC.

```

( 這段游標寫法直接取自 SAP 官方 ABAP 文件 [ABAPEXEC\\_CURSOR](#) 的完整範例，範例本身讀的剛好就是本課程一路用的 [SPFLI](#) 表，這題的答案程式就是照這個官方範例改寫、實際在系統上跑過驗證的。 )

⚠ 一個容易被忽略、但官方文件明講的重點：**Native SQL** 目前基本上是「凍結中」的舊技術——官方文件 [ABENNEWS-740-NATIVE\\_SQL](#) 原文：

New developments in Native SQL are now only possible in ADBC, which means that ADBC is now recommended in new programs instead of the static embedding of Native SQL.

也就是說：這題教的 **EXEC SQL...ENDEXEC** 語法，**SAP** 官方建議新程式不要再用，新功能只往 if08 要教的 **ADBC** ( [CL\\_SQL\\_STATEMENT](#) / [CL\\_SQL\\_CONNECTION](#) ) 發展。這題還是要學，因為既有系統裡大量存在這種寫法 ( 維護舊程式一定會遇到 )，但寫新程式時應該直接跳去用 if08 的 ADBC。這也是這門課刻意把 if06 ( 舊寫法 ) 跟 if08 ( 新寫法 ) 分開兩題、之間夾一題 if07 ( 連線設定 ) 的用意——讓學員先看懂舊的，再學會用新的取代它。

## 學習目標

- 理解 RFC 機制跟 Remote-Enabled Module 屬性的關係 ( 承接 if01 )
- 能寫出 **EXEC SQL...ENDEXEC** 的單筆查詢與游標 ( 多筆 ) 兩種語法
- 理解並能舉例說明「**Native SQL** 不會自動處理 **Client**」，並能對照 Open SQL / AMDP SQLScript，講出這是同一種模式在第三個技術上的展現
- 知道 SAP 官方已經建議新程式改用 ADBC ( if08 )，Native SQL 屬於維護舊系統才需要的技能

## 事前準備

不需要既有物件，這題新建一支唯讀、安全的對照程式 `ZR_IF06_NATIVE_SQL_DEMO`，用 `SPFLI`（航班連線）資料示範 Open SQL 與 Native SQL 讀到的結果一致。

## 題目需求

1. 對照 `ZR_IF06_NATIVE_SQL_DEMO`，指出程式裡「Open SQL 交給框架自動處理」跟「Native SQL 自己手動處理」的是同一件事（Client 過濾）。
2. 如果拿掉 Native SQL 那段的 `WHERE mandt = :sy-mandt`，會發生什麼事？（可以實際試著拿掉、重新啟用執行看看，這套系統的 `SPFLI` 是否也像 AMDP 課程 am01 發現的 `SCARR` 一樣，在多個 Client 灌了重複資料）
3. 解釋為什麼 Native SQL 讀多筆資料要用游標（`OPEN/FETCH/CLOSE`），而不能像單筆查詢一樣直接 `INTO :變數`。

## 答案

見 `zr_if06_native_sql_demo.prog.abap`（SAP 端物件 `ZR_IF06_NATIVE_SQL_DEMO`，套件 `$TMP`）。已在系統上實際啟用並用 `programrun` API 執行驗證：`P_CARRID = LH` 時，Open SQL 與 Native SQL 兩段各自撈出 5 筆（`0400/0401/0402/2402/2407`），內容一致，只有順序不同（Native SQL 那段沒加 `ORDER BY`，這也是故意留著的教學點——Native SQL 的排序跟 Open SQL 一樣需要自己在 SQL 裡明確寫 `ORDER BY`，資料庫沒有義務保證回傳順序）。

## 團隊實務備註

- `EXEC SQL...ENDEXEC` 的語法與游標範例已用 sap-docs 查證 SAP 官方 ABAP 文件（`ABAPEXEC / ABAPEXEC_CURSOR`），游標範例原文剛好就是讀 `SPFLI`，直接照抄改寫成本題答案程式，不是憑空編造語法。
- 官方文件明確說「Native SQL 新開發已經凍結，只能在 ADBC 做」（`ABENNEWS-740-NATIVE_SQL`），這題的定位是「看懂、維護舊程式」，不是「教你在新專案裡首選這個技術」——真的要在程式存取資料庫，Open SQL 能做到的優先用 Open SQL，Open SQL 做不到才考慮 if08 的 ADBC，`EXEC SQL...ENDEXEC` 幾乎只剩「讀懂舊系統遺留程式碼」這個用途。
- 本題程式對 `SPFLI` 只有 `SELECT`，沒有任何寫入，是完全安全、可重複執行的唯讀示範，不會對系統資料造成任何影響。

## 思考題

1. Native SQL 的 `EXEC SQL` 區塊「不會被語法檢查完全檢查」（官方文件原文：the area between EXEC and ENDEXEC is not checked completely by the syntax check）——這代表什麼風險？（提示：一般 ABAP 語句寫錯，啟用時編譯器就會抓到；`EXEC SQL` 區塊裡的內容是直接交給資料庫的原生語法，ABAP 編譯器沒有能力完整檢查，代表語法錯誤可能要等到執行期才會發現，這是 Native SQL 比 Open SQL 更容易在正式環境才炸掉的原因之一）
2. 如果同一段程式要維護（既要能跑在 HANA、又要能跑在其他資料庫），Native SQL 適合嗎？（提示：官方文件另一句話「Programs with Native SQL statements are generally dependent on the database system used」——不適合，這正是 if04 資料轉換技術比較表裡「介面穩定度」一欄提到的可移植性問題，只是這裡換成資料庫可移植性）
3. 這題跟 AMDP 課程 am01「SCARR 不自動過濾 Client」是不是完全一樣的坑？兩者在解決方式上有沒有不同？（提示：問題本質一樣（繞過 Open SQL 框架就要自己處理 Client），解法也幾乎一樣（把 `sy-`

`mandt` / `iv_mandt` 當作條件手動加進 `WHERE` )，差異只在於一個是 `SQLScript` 語法、一個是 `Native SQL` 語法——這代表「Client 過濾」是一個跨技術的通用原則，不是某個特定技術才有的特例 )